Selecting Reliable Mathematical Solutions with Compact Neural and Tree Rankers

DASE7506 - Advanced Machine Learning Methods | Mini-project 2

Final method: v4-wide | Report date: 21 October 2026

Abstract

This project selects one of four supplied mathematical solution traces using only the provided candidate texts and fixed embeddings. The research question is whether explicit local arithmetic evidence and a small complementary tree branch improve a regularized neural selector within a strict CPU and asset budget. The final v4-wide method combines three small neural rankers with 15 depth-4 ranking trees. On validation it selects a correct answer for 752 of 798 questions (94.24%); on the complete test bank it achieves 387/467 (82.87%), compared with 379/467 (81.16%) for the initial linear method. Removing arithmetic evidence and retraining reduces validation accuracy to 93.11%. The tree branch adds only one validation success over the neural branch, so its incremental benefit is modest. The submitted checkpoint uses 2.4475 MiB of uncompressed assets. An independently launched prediction and scoring reproduction takes 5.01 seconds locally, with sampled native thread peaks of four. The method remains a candidate selector, not a proof checker.

1. Task and experimental protocol

Each question supplies four candidate solutions and 1024-dimensional whole-trace embeddings. The output is one Python integer in 0-3, in question order. The official scorer checks the selected candidate's final numeric answer using its fixed parser, including equivalent fractions and decimals. Multiple candidates can be correct; all receive the same credit. Reasoning validity is not itself scored [1].

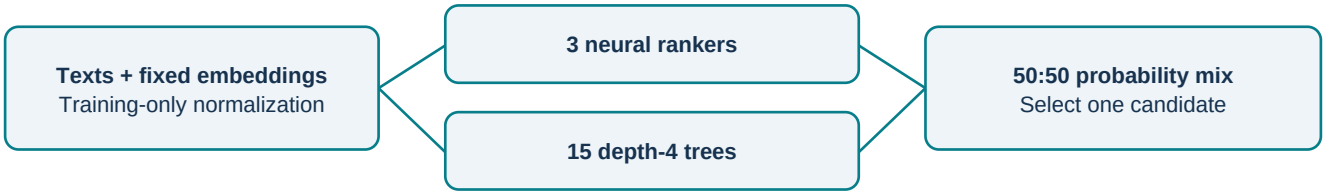
Split	Questions	Role
Training	4,755	Fit weights and preprocessing statistics
Validation	798	Early stopping, configuration selection, analysis
Test	467	Frozen reporting and checkpoint reproduction

The splits and official tools are unchanged. There are 355 validation groups with at least two distinct parseable candidate answers. These disagreement groups are called "hard" below; this definition uses candidate answers, not gold difficulty labels. Every test group has answer disagreement, so the overall validation accuracy is an optimistic description of test difficulty.

The working hypothesis is that arithmetic consistency supplies useful evidence beyond trace style and embeddings, while neural and tree models make partly different errors. This is an empirical design question, not a claim that local calculations establish the correctness of a complete solution. Model development uses training and validation only. No test references enter fitting, fusion selection, features, or inference.

2. Candidate representation and inference

The model processes candidates jointly through relative features, but scores each candidate with shared learned functions. No question ID enters the representation. All normalizers are fitted on training inputs, including easier groups, before they are applied to validation or test.



2.1 Features

For each candidate, the 2168-dimensional input concatenates a normalized 1024-dimensional supplied embedding, its deviation from the within-question mean (1024 dimensions), 60 manual features, and their within-question deviations (60 dimensions). Normalized absolute features are clipped to [-8, 8]. This gives both absolute evidence and a comparison against the other proposed solutions.

Manual features describe trace length, word/number/operator counts, formatting, answer parseability, embedding similarity to other candidates, answer magnitude, question-number and word overlap, repeated-answer support, and question cues such as rates or remainders. These are label-free transformations of the supplied inputs. They can capture superficial shortcuts as well as useful evidence; the ablation does not eliminate all such shortcuts.

Six arithmetic features summarize how many explicit equalities can be checked, how many agree or disagree, the disagreement fraction, whether any disagreement occurs, and the largest relative error. The implementation supports basic arithmetic and a bounded subset of LaTeX fractions. It uses a restricted syntax-tree evaluator with numeric constants and allowed arithmetic operators; it never executes candidate code. Expressions are limited to 160 characters and depth 16, values to magnitude 10^12, and at most 64 checkable equalities per trace. Matching uses relative tolerance 10^-3 and absolute tolerance 10^-5.

Coverage is incomplete: prose, unsupported notation, omitted equations, and the semantic choice of an equation can escape these checks. A candidate may execute its chosen arithmetic correctly while representing the word problem incorrectly.

2.2 Frozen probability combination

Each neural member produces a softmax over the parseable candidates among the four. Its member probabilities are averaged. If c(i) counts candidates sharing candidate i's parsed final answer, the neural probability receives a small consensus adjustment:

```
p_N(i) = normalize( mean_m softmax(s_m)(i) * c(i)^0.1 )
p_T(i) = softmax( tree_score(i) / 1.0 )
p(i)   = 0.5 * p_N(i) + 0.5 * p_T(i)
choice = argmax_i p(i), over parseable candidates
```

The final outer consensus coefficient is zero. The complete deployed weights, statistics and coefficients are stored in one NPZ checkpoint. The predictor reads the passed bank and embeddings, not gold answers or saved predictions. Inference is offline NumPy only.

3. Training and selection

Correct-candidate labels are constructed by comparing parsed training candidate answers against the parsed training reference. Of 4,755 training groups, 2,114 contain distinct valid candidate answers and contribute updates to the improved rankers. Groups where all valid candidates agree do not teach a ranking distinction. Groups with exactly one correct candidate receive a fixed training weight of 1.5; other informative groups receive weight 1.

3.1 Neural branch

Each network has shape 2168 -> 96 -> 32 -> 1, with ReLU hidden activations and dropout after both hidden layers. The three independent seeds are 7506, 7507 and 7508. Training uses a uniform target over all correct candidates, mixed with a uniform distribution over valid candidates using smoothing 0.1. Weighted cross-entropy avoids forcing a single arbitrary positive when several answers are correct.

Setting	Final value
Optimizer	AdamW, learning rate 0.0005
Weight decay / dropout	0.1 / 0.5
Batch size / gradient clipping	128 groups / norm 5
Maximum epochs / patience	80 / 12
Best epochs by seed	16 / 13 / 23
Actual epochs by seed	28 / 25 / 35

An epoch is selected by hard-validation correct count, then overall correct count, then lower marginal positive-set loss. Patience counts epochs without an improvement under this rule. Thus the saved epoch need not be the final epoch. Deterministic CPU training is enabled; dropout is disabled for inference.

3.2 Tree branch and fusion

XGBoost [3] trains a pairwise ranking objective on the same informative training groups and 2168 features. It uses depth 4, learning rate 0.05, subsample 0.85, column sample 0.65, minimum child weight 3, L2 penalty 10, L1 penalty 0.05 and max_bin 128. The seed is 7506. The budget is 800 rounds with patience 60 on hard-validation selection accuracy; the selected model has 15 trees. Exported NumPy tree scores exactly match library scores on the saved validation check (maximum absolute difference 0).

Tree temperature is selected from {0.5, 1, 2}, tree mixture weight from {1, 0.75, 0.5, 0.25, 0}, and outer consensus from {0, 0.1, 0.25}. The primary criterion is hard-validation correct count, followed by overall correct count. Ties across final candidate methods prefer smaller assets. The selected outer coefficients are temperature 1, tree weight 0.5, and consensus 0. The freeze record is dated 14 October 2026, 12:20:47 UTC+8. No parameters were changed after the frozen test result.

4. Development comparisons and final accuracy

The initial v1 method is a linear listwise ranker with a fixed random embedding projection and manual features, plus validation-selected answer consensus. It uses the same supplied data and official scorer. The improved neural variants add the richer representation and stronger regularization. The table reports validation selection results, not independent repeated-test comparisons.

Method	Validation	Hard validation
Initial v1 linear + consensus	92.98%	84.23%
v2 neural ensemble	93.48%	85.35%
v3, smaller regularized neural	93.73%	85.92%
v4, depth-4 trees + small v3	93.86%	86.20%
v4, depth-2 tree alternative	93.86%	86.20%
v3-wide, three 96-unit members	94.11%	86.76%
Six regularized neural members	94.11%	86.76%
Nine mixed neural members	93.98%	86.48%
Final v4-wide	94.24%	87.04%
Trees + six neural members	94.24%	87.04%

The larger ensembles do not improve validation accuracy. The final three-member combination is preferred over the equally accurate six-member combination because it is smaller. This supports regularization and limited model complementarity rather than simply increasing ensemble size. All search candidates and their source checkpoint records are retained in the freeze manifest and search reports.

Official metric	Initial v1	Frozen v4-wide	Change
Validation correct / 798	742	752	+10
Hard-validation correct / 355	299	309	+10
Complete test correct / 467	379	387	+8
Complete test accuracy	81.16%	82.87%	+1.71 pp

The exact test accuracy field to submit is 82.86937901498929. The final model makes 80 test errors. On paired validation choices it fixes 18 v1 errors and introduces eight new errors, giving the net increase of ten. On the 117 validation groups with exactly one correct candidate, accuracy rises from 73/117 (62.39%) to 80/117 (68.38%). This difficult diagnostic is still far below the overall validation score.

The final model was tested after freezing. The initial baseline test score had already been observed before later development, so this sequence should not be described as a fully blind, preregistered comparison. The code uses no test feedback for fitting or configuration search. Repeated use of validation for development creates selection bias, and small numerical gains are not evidence of statistical significance.

5. Ablation evidence and failure analysis

Only the final selected method receives the reported ablations. All are evaluated on validation; no ablation is scored on test.

Final-method variant	Correct / 798	Validation	Hard validation
Full frozen v4-wide	752	94.24%	87.04%
Remove tree, keep frozen neural	751	94.11%	86.76%
Remove neural, keep frozen tree	734	91.98%	81.97%
Remove arithmetic and retrain	743	93.11%	84.51%

The two branch-deletion experiments reuse frozen weights and select the remaining branch without retuning. The arithmetic ablation disables the six arithmetic features and their relative counterparts, refits normalization and retrains both branches using the same architectures, seeds, budgets and early-stopping rules. Fusion coefficients remain fixed to the complete method. Best epochs and tree count may differ because the same early-stopping rule is reapplied. This is a retraining ablation, not merely zeroing inputs to the original weights.

Removing arithmetic loses nine correct validation choices (1.13 percentage points), which supports its usefulness under this protocol. The result does not isolate a full-reasoning advantage over every possible final-answer-only model: other text features and embeddings remain. Removing the neural branch loses 18 correct choices, while removing trees loses only one. The neural branch provides most of the performance; evidence for the extra tree branch is weak and may not generalize.

5.1 What the remaining errors show

There are 46 saved validation errors. The following examples are selected from that file for diagnosis, not used to revise the frozen method.

Semantic bookkeeping. A square-dancing question combines classes with 46 male and 48 female students. The selected trace computes 46 possible pairs but then subtracts 46 from 94 students, producing 48 instead of the reference answer 2. Its displayed subtraction is arithmetically correct. The mistake is failing to count two students per pair, beyond a local equality check.

Omitted state updates. In a pie question, four pies are reduced by one sale and one gift. The selected trace incorrectly carries three pies forward, then consistently computes 18 pieces, 12 eaten and six left. The reference is four pieces. Later correct arithmetic cannot repair the earlier state error, especially when the missing transition is expressed only in prose.

Numerical reasoning and notation. A pursuit question uses a 0.6-mile head start and a 9-mph relative speed. The selected trace incorrectly converts 0.6/9 hours to 2/15 hours, returning eight minutes rather than four. This illustrates why arithmetic evidence can help, while the remaining failure also shows that partial equation extraction and learned ranking do not guarantee rejection of every faulty trace.

These cases motivate future semantic or quantity-tracking evidence, but that would require new validation and a new frozen evaluation protocol. It is not added to this submission.

6. Resources, integrity and reproducibility

The coursework allows at most 16 MiB of extra uncompressed inference assets and 30 seconds for complete CPU prediction plus independent scoring, using four threads in total [1]. Supplied data and ordinary source code are excluded from the asset budget. All inference parameters and statistics are included in the single checkpoint; no external weights are required.

Resource	Measured value
Raw array payload	2,560,608 bytes
Uncompressed NPZ members, including NPY headers	2,566,368 bytes = 2.4475 MiB
Checkpoint file size	2,572,508 bytes
Local CPU / environment	Intel Core i7-9750H / Windows 11, mp1
Runtime libraries for prediction	Python 3.12.14, NumPy 2.5.3
Audited fresh prediction + scorer	4.3970 + 0.6096 = 5.0066 seconds
Sampled native thread peaks	Prediction 4, scoring 4; sequential

The audited reproduction was run from the submitted folder on 21 October 2026 with all relevant library thread caps set to 1. Operating-system sampling records four live threads per evaluation process. The timer includes process startup, imports, input and asset loading, feature extraction, every ensemble member, independent scoring and monitoring overhead. The original 14 October run recorded 3.9656 seconds with library limits of 4; it is preserved as historical evidence rather than used to certify the aggregate native thread limit. The new run reproduces every one of the 467 original test choices with unchanged weights and coefficients. It is checkpoint reproduction, not a new tuning run. Eligibility on the specified Xeon Platinum 8457C remains subject to course measurement.

Search training and fusion scripts recorded approximately 247.34 seconds internally, excluding imports. The arithmetic retraining ablation adds 68.48 seconds internally. Whole subprocess wall times and commands are retained separately; installation costs are not included in those training totals. These numbers do not estimate all human development time or every diagnostic command.

Independent from-scratch reproduction of the final training pipeline took 106.01 seconds including process imports. It recreated exactly the same model file, all 45 arrays and all 798 validation choices. This verification used training and validation only. The supplied embeddings are fixed, and the learned rankers are small; short training is expected and does not mean a language model was trained from scratch.

The frozen checkpoint SHA-256 is:

```
508b155570e105f005ecc1dc7f52c024bbb9
3b69daa45bcf1259fb04d8b17d41
```

The two displayed lines concatenate to one 64-character hash. Frozen source hashes, data hashes, official scores, prediction choices, error examples and independent reproduction records are distributed with the code. Input-size, arbitrary-ID, integer-output and candidate-permutation checks are also provided.

7. Reproduction, limitations and disclosure

7.1 Running the submitted version

The package version is mp2-v4-wide-final-20261021; its file manifest identifies exact source bytes and the matching checkpoint. Activate mp1 and set OMP_NUM_THREADS, OPENBLAS_NUM_THREADS, MKL_NUM_THREADS and NUMEXPR_NUM_THREADS to 1. From the code directory:

```
python -B verify_package.py
python -B measure_cpu.py --split test --output runs/reproduced_test
python -B run_logged.py --log runs/retrained.log -- train_best.py
    --output runs/retrained
```

The last command is one line in the root README, which supplies full CMD commands. Evaluation needs only NumPy and the checkpoint. Training adds PyTorch 2.7.1 [4], XGBoost 3.1.3 and SciPy 1.18.1. The original PyTorch build was 2.7.1+cu126; all training used CPU. Every learned transformation and mixture coefficient is saved.

7.2 Limits of the evidence

The scorer assesses final answers, not step-by-step validity. Embeddings and manual cues may learn formatting or dataset-specific associations. The arithmetic ablation supports a narrow claim about the included features; it does not demonstrate general mathematical verification. Repeated validation selection, a small hard-validation subset and a single fixed test bank limit generalization claims. The tree branch's one-question validation gain does not justify describing it as a robust improvement. No calibration study, confidence guarantee, or audit-bank result is claimed.

Within this scope, v4-wide improves the initial accuracy within the resource limits. The arithmetic retraining contrast and difficult single-correct subset provide useful evidence alongside the remaining failures. More computation alone did not improve selected validation accuracy.

7.3 AI assistance and reused work

Substantial OpenAI Codex assistance was used for feature and model design, training and inference implementation, experiment automation, result analysis, reproducibility packaging, and drafting this report. The submitting student is responsible for understanding and explaining the work. The course supplied the dataset, fixed representations, runner, parser, scorer and guide. The implementation uses NumPy, PyTorch and XGBoost; no external training data, additional pretrained weights, API generation, answer lookups by ID or cached predictions are used by the selector. Saved predictions are evidence only.

GSM8K [2] supplies the source questions/references; its MIT license and notice are retained. Course-generated traces and embeddings retain their manifest provenance. Publish code and checkpoint links with a fixed commit/tag and enter the student ID through the course website.

References

[1] DASE7506 course staff. Mini-project 2: Selecting Reliable LLM Reasoning. Fall 2026. Included guide/GUIDE.md and code/OFFICIAL_README.md; https://github.com/xudongwu-0/DASE7506-MP2.

[2] Cobbe, K. et al. Training Verifiers to Solve Math Word Problems. 2021. arXiv:2110.14168. GSM8K data source: https://github.com/openai/grade-school-math.

[3] Chen, T. and Guestrin, C. XGBoost: A Scalable Tree Boosting System. KDD, 2016. doi:10.1145/2939672.2939785.

[4] Paszke, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS, 2019.